# Clases Grafos Algoritmos

## Vínculos

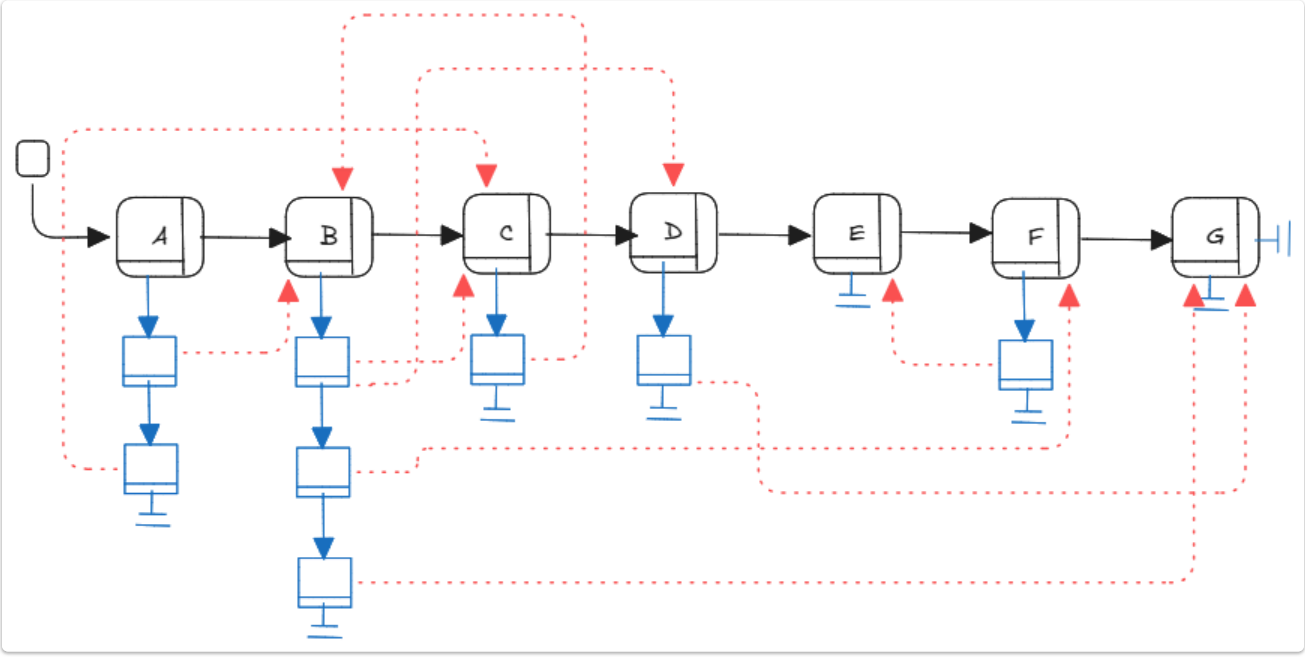
- Clase: Algoritmos 2 Clase 6
- Definición de clases: Clases Grafos Algoritmos
- Ejercicios: Ejercicios Grafos

## Representación en memoria

### Lista de adyacencia

Funciona con dos listas, una lista de los nodos vertices (nodos conectados linealmente) donde cada vértices tiene una lista de apuntadores a otros vértices (una lista de adyacencia).

- Es útil en grafos no densos
- **En memoria**: $O(n+m)$ donde $n = |V| \quad m = |A|$




## Clases Nodos

### Nodo Arco

```
Class NodoArco
    Attributes
        Private:
            pointer to NodoVertice<Element>: info
            pointer to NodoArco<Element>: siguiente
            float: peso
    Methods
        Public:
            proc construir(...)
            // Getters
            ...
            // Setters
            ...
endClass
```

### Nodo Vértice

```
Class NodoVertice
    Attributes
        Private:
            Element: info
            pointer to NodoVertice<Element>: siguiente
            pointer to NodoArco<Element>: listaAdy
    Methods
        Public:
            proc construir(Element: info, pointer to NodoVertice<Element>: sig, pointer to
NodoArco<Element>:listaAdy)
            // Getters
            ...
            // Setters
            ...
endClass
```

## Clase Grafo

> *En métodos como el DFS se utiliza un método interno privado recursivo de igual nombre con diferentes parámetros (polimorfismo).*
> *De manera similar, los métodos públicos recursivos requieren un método privado, ya sea polimórfico o no.*

```
Class Grafo
    Attributes
        Private:
            pointer to NodoVertice<Element>: g
            int: nVertices, nArcos
            bool: esDirigido
    Methods
        Public:
            proc constructor(...)
            // Getters
            ...
            // Setters
            ...

            func getVertices(): Lista<Element> // O(n)
            func getArcos(): Lista<Lista<Element>> // O(n + m)
            func getPesoArco(Element: v, w): float // O(n + arcos(v))
            func getSucesores(Element: v): Lista<Element> // O(n+ arcos(v)) Solo en dirigidos
            func getPredecesores(Element: v): Lista<Element> // O(n + m)
            func getGradoVertice(Element: v): int // O(n + arcos(v))
            func getVecinos(Element: v): Lista<Element> //O(n + arcos(v)) No dirigidos
            proc insertarVertice(Element:v) // O(1) Se inserta al principio
            proc insertarArco(Element: v, w, float: peso) // O(n)
            proc eliminarVertice(Element: v) // O(n + m)
            proc eliminarArco(Element: v, w) // O(n + arcos(v) + arcos(w))
            proc setPeso(Element: v, w, float: peso) // O(n + arcos(v) + arcos(w))
            func bfs(): Lista<Element> // O(n + m)
            func dfs(): Lista<Element> // O(n + m)
endClass
```